

Sie dürfen alle Ihre Unterlagen, alte Programme und Internetquellen verwenden.
Kommunikation oder Austausch in jedweder Art (analog oder digital) ist nicht erlaubt.

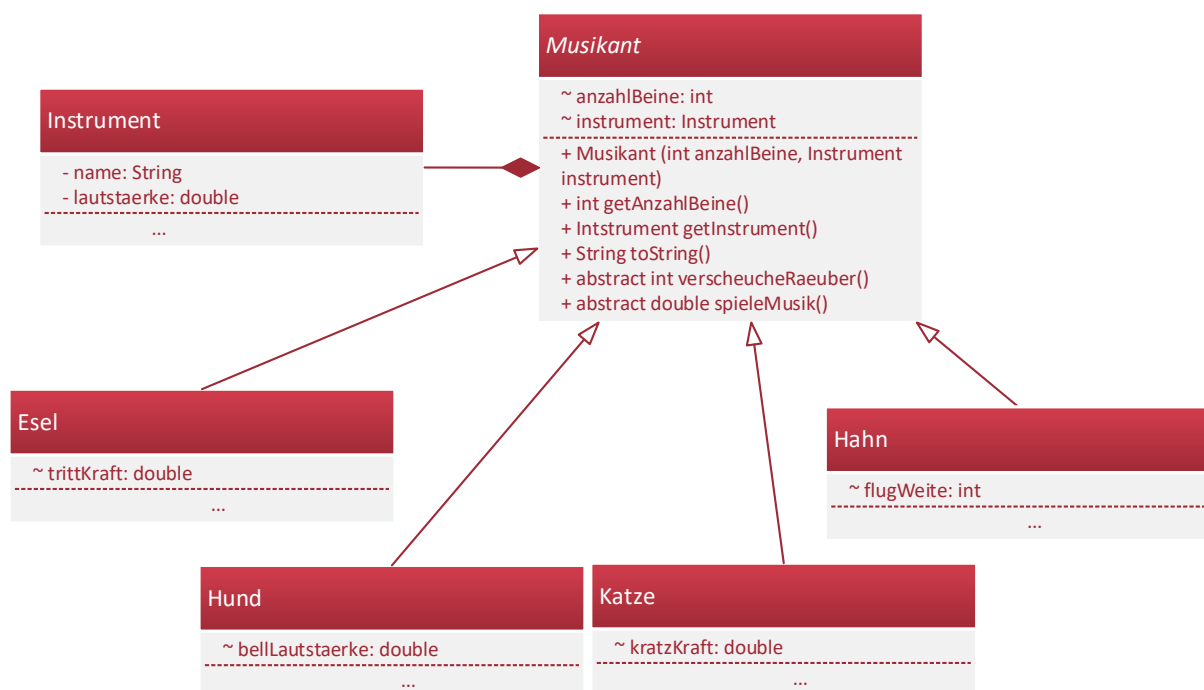
Über Moodle haben Sie Zugriff auf ihr Startpaket. Bitte geben Sie am Ende der Zeit Ihr Projektverzeichnis als ZIP Datei über Moodle ab.

Nur für compilierbaren und funktionsfähigen Code erhalten Sie Punkte! Die Junit Tests sind nur zur Hilfe und Orientierung für Sie und garantieren keine vollständige Korrektheit Ihres Codes. (Diese sind anfangs auskommentiert, damit Ihnen dort keine Fehler angezeigt werden.)

Halten Sie sich an die Angaben: Keine Veränderung von Methodennamen oder Struktur, Textausgabe statt return Wert, ...

Die Bremer Stadtmusikanten (53 Punkte)

Eine grobe Übersicht der wichtigsten Klassen sehen Sie im folgenden Klassendiagramm:



Für einige Klassen gibt es teilweise Implementierungen oder leere Klassenvorlagen – erweitern Sie diese.

[1 Punkt] Erstellen Sie für die Klasse **Instrument** einen Konstruktor zur Initialisierung der Membervariablen und erstellen Sie getter und setter Methoden für die beiden Variablen.

[2 Punkte] Erstellen Sie eine abstrakte Klasse **Musikant** und fügen Sie die beiden Eigenschaften aus dem Klassendiagramm hinzu. Implementieren Sie den Konstruktor zum Initialisieren (der beiden Eigenschaften) und die beiden get Methoden aus dem Klassendiagramm.

[1 Punkt] Erstellen Sie in der Klasse **Musikant** die beiden abstrakten Methoden aus dem Klassendiagramm:

[2 Punkte] Überschreiben Sie die **toString** Methode und erstellen Sie einen String der die Rückgabewerte der abstrakten Methoden *verscheucheRaeuber* (Rückgabewert *1*) und *spieleMusik* (Rückgabewert *2*) auf folgende Art aufbaut:

Verscheucht:*1*, Musiziert:*2* (statt *1* bzw. *2* die Rückgabewerte einsetzen)

z.B: Verscheucht: 4, Musiziert: 6.0

[4 Punkte] Erstellen Sie die Klassen **Esel**, **Hund**, **Katze** und **Hahn**, die sich von der abstrakten Klasse Musikant ableiten. Erstellen Sie für jede Klasse Konstruktoren um die Membervariablen der Basisklassen und die neuen Membervariablen zu initialisieren. (Zuerst die Membervariablen der Basisklasse) Erstellen Sie für die neuen Membervariablen getter.

[2 Punkte] Überschreiben Sie für die Klassen **Esel**, **Hund**, **Katze** und **Hahn** die **toString** Methode und fügen sie **zu Beginn des Strings** den Namen der Klasse und den Wert der neuen Membervariable (der abgeleiteten Klasse) hinzu und danach den bestehenden String aus der Basisklasse. (Verwenden Sie die Implementation der Basisklasse)

Name der Klasse *Membervariable*: *String von Musikant toString*

z.B: Esel 2.3: Verscheucht: 4, Musiziert: 6.0

Implementieren Sie die **verscheucheRaeuber** Methoden – Diese liefern als Rückgabewert wie viele Räuber ein Musikant verscheuchen kann. Runden und casten Sie den Rückgabewert mittels **(int) Math.floor (...)** ab:

- **[1 Punkt] Esel:** Der Esel kann Trittkraft * Beinanzahl viele Räuber verscheuchen.
- **[2 Punkte] Hund:** Der Hund verscheucht die Räuber mit lauten Geräuschen und verwendet je nachdem was lauter ist entweder sein Bellen oder sein Instrument.
- **[3 Punkte] Katze:** Die Katze kratzt die Räuber und verscheucht sie damit. (z.B. Kraft 3.8 verscheucht 3 Räuber) Wenn die Katze verletzt ist und nur 3 Beine hat kann sie nur halb so stark kratzen und wenn die Katze nur 2 Beine (oder weniger) hat kann sie nur 1 Räuber verscheuchen.
- **[4 Punkte] Hahn:** Wie viele Räuber der Hahn verscheucht ist abhängig von seiner Flugweite und der Lautstärke seines Instruments.
 - o Bei jeder Flugweite kleiner als 2 verscheucht er Lautstärke viele Räuber.
 - o Bei Flugweiten zwischen 2 und 6:
 - Flugweite 2 -> 6 Räuber
 - Flugweite 3 -> 5 Räuber
 - Flugweite 4 -> 4 Räuber
 - Flugweite 5 -> 3 Räuber
 - Flugweite 6 -> 2 Räuber
 - o Bei jeder Flugweite größer als 6 verscheucht er 1 Räuber.

(int) Math.floor bei jedem Tier nicht vergessen!

Implementieren Sie die **spieleMusik** Methoden – der Rückgabewert liefert die Lautstärke.

- **[1 Punkt] Esel** und **Katze** sind so laut wie ihr Instrument.
- **[1 Punkt] Die Hund** Lautstärke ist der mittlere Wert zwischen Instrument- und Bellautstärke. (Lautstärke Bellen 10, Lautstärke Instrument 20 -> Ergebnislautstärke ist 15)
- **[1 Punkt] Der Hahn** kräht zusätzlich zu seinem Instrument, weshalb sich seine Instrumentlautstärke um 2 erhöht. Da er auch noch durch die Luft fliegt wird seine Lautstärke durch die Flugweite dividiert.

[2 Punkte] Erstellen Sie eine Klasse **Quartett**, die Musikanten in einer privaten ArrayList speichern kann. (Wählen Sie selbst einen Namen) Erstellen Sie einen Konstruktor um diese zu initialisieren. Erstellen Sie weiters eine **void add(Musikant m)** Methode um einen Musikanten in die ArrayList einzufügen.

Die restlichen Punkte können Sie in beliebiger Reihenfolge implementieren:

[1 Punkt] Erstellen Sie in der Klasse **Quartett** eine Methode **boolean istQuartett()** die überprüft ob das Quartett vollständig ist. (= enthält 4 Mitglieder)

[4 Punkte] Erstellen Sie in Klasse **Quartett** eine Methode **int gemeinsamRaeuberVerscheucht()** die aufsummiert wie viele Räuber durch das Quartett verscheucht werden können.

[zusätzlich 2 Punkte] Verwenden Sie in dieser Methode einen **Iterator**.

[4 Punkte] Erstellen Sie in Klasse **Quartett** eine Methode **double durchschnittlicheLautstaerke()**, welche die durchschnittliche Lautstärke des Quartetts als Rückgabewert liefert. (Die Methode `spieleMusik` verwenden)

[5 Punkte] Erstellen Sie in der Klasse **Quartett** folgende Methode **ArrayList<Musikant> getMusikantenInLautstaerkeBereich(double von, double bis)** die in einer Liste alle Musikanten zurückliefert deren Lautstärke (`spieleMusik`) zwischen den beiden Grenzen liegt.

[5 Punkte] Erstellen Sie in der Klasse **Quartett** folgende Methode **HashMap<Integer, Integer> getAnzahlMusikantenMitBeinAnzahl()** die zählt wie viele Musikanten es jeweils mit einer spezifischen Nummer an Beinen gibt. In der HashMap sind die Schlüssel die Anzahl der Beine und der Wert die Anzahl der Tiere mit entsprechend vielen Beinen. (z.B. Eintrag **2 - 3** – **Schlüssel** 2 steht für 2 beinige Tiere; Wert 3 gibt an, dass 3 Tiere entsprechend viele Beine haben)

[5 Punkte] Erstellen Sie einen Comparator mit dem Namen **LautstaerkeComparator** der Musikanten sortieren kann. Die Musikanten sollen **ABSTEIGEND** nach ihrer Lautstärke (-> `spieleMusik`) sortiert werden. Erstellen Sie in der Klasse **Quartett** eine Methode **void printLautStaerkeAbsteigend()** die die Liste sortiert und jeden Musikanten ausgibt. (`toString` verwenden)

Im **MusikantenTest** können Sie die Funktionalität Ihrer Klassen testen.